AWS Lambda - Service Overview

Introduction

AWS Lambda is a serverless, event-driven compute service that lets you run code without provisioning or managing servers. Lambda automatically scales your application by running code in response to triggers, and you are billed only for the compute time actually consumed, measured in milliseconds.

Execution Model

A Lambda function is deployed as a package (a ZIP archive or a container image) containing your code and dependencies. When triggered, Lambda provisions an execution environment, runs your handler function, and returns the result. If no invocations occur for a period, Lambda deprovisions the environment; the next invocation after idle time incurs a "cold start," where a new execution environment must be initialized before your code runs.

Triggers

Lambda functions can be invoked by a wide range of event sources, including:

- API Gateway (for building REST or HTTP APIs)
- S3 (e.g. triggering on object upload)
- DynamoDB Streams (reacting to table changes)
- SQS and SNS (processing queued or published messages)
- CloudWatch Events / EventBridge (scheduled or rule-based invocation)
- Direct SDK invocation from another application

Configuration Limits

Lambda functions have configurable memory (128 MB to 10,240 MB), with CPU allocation scaling proportionally to memory. Maximum execution timeout is 15 minutes. Deployment package size is limited to 50 MB (zipped, direct upload) or up to 10 GB for container images.

Concurrency

By default, Lambda automatically scales the number of concurrent executions to meet incoming request volume, up to an account-level concurrency limit (which can be raised via a service quota request). Reserved concurrency can be set on a specific function to guarantee it a portion of the account's concurrency pool, or to throttle it to prevent it from overwhelming a downstream resource like a database.

IAM Execution Role

Every Lambda function has an associated execution role, granting it permission to interact with other AWS services (for example, writing logs to CloudWatch, reading from an S3 bucket, or writing to a DynamoDB table). As with ECS task roles, this should be scoped to only the specific actions and resources the function needs.

Layers

Lambda Layers allow you to package and share common dependencies or libraries across multiple functions, reducing the size of each individual deployment package and simplifying dependency management.

Cost Model

Lambda pricing is based on the number of requests and the duration of code execution, billed per millisecond and rounded up, multiplied by the amount of memory allocated to the function. There is a perpetual free tier covering 1 million requests and 400,000 GB-seconds of compute per month.

Cold Starts and Provisioned Concurrency

A cold start occurs when Lambda must initialize a brand new execution environment before running your handler, which includes downloading the deployment package, starting the runtime, and running any code outside the handler function. Cold start latency varies by runtime and package size, ranging from tens of milliseconds for lightweight interpreted runtimes to a few seconds for large container-image-based functions with heavy dependencies. For latency-sensitive applications, Provisioned Concurrency keeps a specified number of execution environments pre-initialized and ready to respond immediately, eliminating cold starts for that reserved capacity, at an additional ongoing cost even when the function is not actively invoked.

Lambda vs Fargate for Long-Running Work

Because of the 15-minute maximum execution timeout and the cold-start characteristics described above, Lambda is not well suited to long-running or steady, high-throughput background processing, such as a continuously polling ingestion worker for a RAG pipeline. For that kind of sustained workload, an ECS Fargate task (or a long-lived EC2 process) that stays warm and processes a queue continuously is typically both cheaper and simpler than orchestrating repeated short Lambda invocations.

Common Use Cases

- Backend logic for lightweight REST APIs behind API Gateway
- Image or file processing triggered by S3 uploads
- Scheduled maintenance tasks (cron-like jobs via EventBridge)
- Glue logic connecting multiple AWS services in an event-driven architecture

Best Practices

- Keep functions small and focused on a single responsibility to minimize cold start time and simplify testing.
- Avoid initializing expensive resources (like database connections) inside the handler function; initialize them outside the handler so they persist across warm invocations.
- Set an appropriate timeout and memory allocation based on actual observed execution profiles rather than defaults.
- Use Lambda for spiky, event-driven, or infrequent workloads; for sustained, high-throughput, long-running workloads, container-based compute (ECS/Fargate) is often more cost-effective.